BEL WIC PROJECT

Remote HSIT Automation Using a Jenkins Agent

Manager briefing • Architecture • Network reasoning • Step-by-step proof of concept

Objective
Keep the existing Jenkins/LDRA analysis workflow at the development location, while running hardware-dependent tests on a dedicated Windows laptop or PC placed beside the WIC target at the factory.

Recommended starting design
A dedicated Windows factory machine runs a Jenkins inbound agent. Jenkins schedules a job on that agent; the agent operates the locally connected target through the approved debugger/test interface and returns logs and results to Jenkins.

Prepared as a discussion and learning guide. Final implementation depends on BEL network, security, tool licensing, test-plan and certification requirements.

1. What to tell the manager

Suggested opening explanation

> “Sir, I studied the Jenkins Agent approach. I recommend keeping our current Jenkins Controller and LDRA automation at our development location, and placing a dedicated Windows laptop/PC beside the WIC hardware at the factory. The factory machine would run a Jenkins inbound agent. Jenkins would assign only the hardware-dependent HSIT job to that agent. The agent would execute the approved test procedure using the local debugger/interface, collect results, and return the evidence to Jenkins. Before implementation, we need to confirm the allowed network path, the target connection method, and how the approved software revision will be transferred.”

Project context

1. **Existing work:** Jenkins orchestrates LDRA static analysis and unit testing on the development-side Windows environment.
1. **Next requirement:** Automate the hardware-dependent part of HSIT with access to the actual WIC target at the factory.
1. **Potential approach:** Use a factory-side Jenkins Agent rather than trying to control all target hardware directly from the development server.
1. **Important boundary:** The agent is an execution machine, not a test tool by itself. CCS/debugger drivers, approved test scripts, target interface and result checks must still be available and configured.

What success looks like

1. Jenkins can reliably schedule a test on the factory agent.
1. The factory agent can identify and access the intended target using the approved hardware interface.
1. A specific approved test case runs and its observed result is compared with an expected result.
1. Logs and metadata identify the tested software revision, test case, configuration and outcome.
1. Failures, unavailable targets and unexecuted tests are reported as failures/blocked—not silently treated as passes.

Terminology

Term	Meaning in this design
Jenkins Controller	The existing Jenkins server. Schedules jobs, manages nodes, records build status and coordinates the pipeline.
Agent / node	A separate machine that runs the assigned build or test steps. The factory laptop is the proposed agent machine.
Executor	A slot allowing one job to run on an agent. Start with one executor so jobs cannot operate the same target simultaneously.
Label	A tag such as wic-hsit used to direct only hardware-specific jobs to the factory agent.
Target	The physical WIC hardware under test; it is not the Jenkins Agent.

2. End-to-end architecture

Recommended logical design

Location	Component	Responsibility
Development location	Existing Jenkins Controller	Orchestrate the pipeline, schedule jobs, retain console output and publish reports.
Development location	SVN + LDRA environment	Checkout a known revision; run existing static analysis and those unit tests that do not require factory hardware.
Approved transfer boundary	Revision/package hand-off	Make the exact approved source/build/test inputs available to the factory workflow, using a method approved by BEL IT/security.
Factory location	Dedicated Windows laptop/PC + Jenkins Agent	Run the assigned HSIT job locally; use installed tools and collect logs. Keep the machine dedicated to testing where possible.
Factory location	WIC target + debugger/interface	Execute the real hardware-dependent test procedure and provide the signals/observations needed for verification.
Development location	Central report/archive	Combine available evidence; identify revision, configuration, test outcomes, failures and missing evidence.

Pipeline flow

1. Select/record SVN revision → 2. Checkout and run existing LDRA checks → 3. Package approved software and test inputs → 4. Schedule HSIT stage on label **wic-hsit** → 5. Factory agent runs test using the local target interface → 6. Collect logs/results → 7. Generate consolidated report and notify reviewers.

Why a factory laptop can work

1. The laptop is physically close to the target and can use its local USB/JTAG/debugger or other approved interface.
1. Jenkins only needs a permitted connection to the agent to schedule and monitor work; the controller does not necessarily need a direct connection to every hardware interface.
1. The laptop can be a proof-of-concept machine first. For permanent operation, check power, automatic restart/reconnection, thermal conditions, physical security, maintenance, licensing and whether a dedicated industrial/desktop PC is preferred.

What the agent does NOT do automatically

1. It does not create network routes or bypass subnet/firewall restrictions.
1. It does not make an Ethernet-only target reachable if routing or target configuration is missing.
1. It does not replace CCS, debugger drivers, test instruments, test scripts or test oracle/expected-result checks.
1. It does not prove HSIT passed merely because the agent is online or a target responds to ping.

3. Why pinging is not a complete solution

Your original answer—“if there is an Ethernet connection, we can ping the hardware and perform testing”—is partly right. Ping can be a useful first diagnostic, but it is not enough to design or prove remote test automation.

Check	What it tells us	What it does not prove
Ping / ICMP	A ping reply suggests ICMP can travel between the two endpoints at that moment.	It does not prove Jenkins can connect, the required TCP/HTTPS port is open, or the target test interface works.
Jenkins Agent connection	The factory machine can authenticate and maintain the configured connection to Jenkins.	It does not prove CCS/debugger can control the target or that a test case passes.
Debugger / target discovery	The factory PC can identify/connect to the target through the configured interface.	It does not prove application behavior is correct.
Actual HSIT case + expected-result check	The specified operation ran and its observed result was compared with the approved expectation.	Only the defined test scope is covered; it does not automatically prove every certification requirement.

Why ping may mislead

1. **Different subnets:** Communication depends on routing and firewall policy. Separate subnets can communicate if an approved route and rules exist; they are not automatically connected.
1. **ICMP may be blocked:** Ping can fail even while the required application connection works.
1. **Ping can succeed while the service is blocked:** ICMP may be allowed but the Jenkins/HTTPS connection or another required port may be denied.
1. **Target may not use Ethernet:** The WIC board may be controlled over JTAG/USB through CCS or another debugger. In that setup, the laptop communicates with the debugger/target locally, and the target itself may not have an IP address.
1. **Connectivity is not correctness:** A reachable board can still fail a test, return an incorrect value, or fail to meet timing/functional expectations.

A better explanation to the manager

> “Ping is only one network diagnostic. We need to test three layers separately: (1) approved connectivity from the factory laptop to Jenkins, (2) local access from the laptop to the target through the actual debugger or network interface, and (3) execution of the approved HSIT case with its expected result checked. A Jenkins Agent addresses where the job executes; it does not itself solve routing or prove hardware behavior.”

Simple troubleshooting order

1. From the factory laptop, open the approved Jenkins URL in a browser.
1. Ask the network team which connection path and ports are allowed; do not assume a port number.
1. Check the agent log and Jenkins node status.
1. Separately use the approved target/debugger utility to confirm target access.
1. Run a harmless diagnostic, then one approved test case and verify its result.

4. Step-by-step Jenkins Agent setup from scratch

Use this as a controlled proof of concept. Perform it with the Jenkins administrator and network/security team. The exact labels and launch-method wording can differ by Jenkins version and installed plugins.

Step 0 — Confirm prerequisites

1. Permission to add a permanent Jenkins node and create/modify a test job.
1. A dedicated factory Windows laptop/PC with a stable network connection and an approved route to the Jenkins Controller.
1. A Java runtime version supported by your Jenkins release, installed and verified on the laptop.
1. A dedicated local workspace, e.g. `C:\JenkinsAgent`, with sufficient disk space.
1. Agreement on the connection method (prefer inbound agent over HTTPS/WebSocket if the installed Jenkins supports it and security/network policy permits it).
1. A defined target interface and approved local test tools. Do not begin by running the real hardware test.

Step 1 — Confirm the Jenkins URL and network path

1. On the factory laptop, open the same approved Jenkins URL users normally use.
1. If it does not open, ask IT to confirm DNS, routing, proxy requirements and firewall rules. Do not work around company controls.
1. If HTTPS is used, confirm the certificate is trusted by the laptop. Do not disable certificate validation as a workaround.
1. The Jenkins Agent does not need the laptop to accept an unsolicited inbound connection if the selected inbound-agent design has the laptop initiate the connection; the required outbound path must still be permitted.

Step 2 — Create the node in Jenkins

1. Log in with an account authorized to administer nodes.
1. Navigate to **Manage Jenkins** → **Nodes** (sometimes shown as Manage Nodes and Clouds).
1. Choose **New Node**, enter `WIC-HSIT-Agent`, select a permanent node/agent, then create it.
1. Configure the fields below, then save. Do not reuse an unrelated agent.

Field	Suggested proof-of-concept value	Notes
Node name	WIC-HSIT-Agent	Use a clear, unique name.
Remote root directory	C:\JenkinsAgent	Create it on the factory laptop; ensure the agent service account can write there.
Labels	wic-hsit windows	Pipeline will target wic-hsit.
Executors	1	Prevents simultaneous jobs from competing for one physical target.
Usage	Only build jobs with matching labels	Reduces the risk of unrelated jobs running on the factory PC.
Launch method	Inbound agent / connect agent to controller	Use the method offered by this Jenkins installation; prefer WebSocket over HTTPS if supported and approved.

4. Step-by-step setup (continued)

Step 3 — Start the inbound agent on Windows

1. Open the newly created node page in Jenkins and read its displayed agent connection instructions.
1. Download the agent JAR from the link Jenkins provides on that page (or the administrator-approved Jenkins agent JAR URL).
1. Save it in a permanent location, for example `C:\JenkinsAgent\agent.jar`.
1. Install/verify the supported Java runtime. In Command Prompt, run `java -version`.
1. Use the exact command generated by Jenkins for this node. It includes the controller URL, node name and a secret. If the WebSocket option is offered and approved, select/use it as documented by your Jenkins version.
1. Do not copy a real agent secret into a report, email, source repository or shared chat. If the secret is exposed, rotate/reconfigure it with the administrator.

Command shape only — not a command to copy unchanged

```
java -jar C:\JenkinsAgent\agent.jar -url https://YOUR-JENKINS-URL/ -secret <SECRET-FROM-JENKINS> -name WIC-HSIT-Agent
```

Important: The actual command and flags must come from the node page or the documentation for your installed Jenkins version. The example contains placeholders; do not type the placeholder secret. If WebSocket is not supported or permitted, the administrator must select another approved agent connection method and open only the required network path.

Step 4 — Verify the agent

1. Run the command in a Command Prompt on the factory laptop.
1. Return to the node page. It should show the agent as online/connected.
1. Review the node log if it remains offline. Check URL, Java version, clock/certificate trust, network/firewall policy and the exact generated command.
1. At this point, only agent connectivity has been demonstrated—not hardware access.

Step 5 — Keep the agent available after reboot

1. For a temporary proof of concept, run the command interactively and verify it stays connected.
1. For a permanent unattended setup, have IT/admin approve a service or Windows Task Scheduler startup task that runs under a dedicated least-privilege account.
1. Configure restart/retry behavior, start-at-boot, and appropriate logging. Make sure the account can access the workspace, CCS/debugger tools and required device drivers.
1. Disable sleep/hibernation only with IT approval and according to the factory operating procedure. Provide approved power backup if required.

Step 6 — Run a harmless test job

Create a new temporary Pipeline job (do not edit the working WIC pipeline yet). Use the label that was assigned to the agent.

```
pipeline {
    agent { label 'wic-hsit' }
    stages {
        stage('Verify Factory Agent') {
            steps {
                bat 'hostname'
                bat 'java -version'
                bat 'echo Agent workspace test'
            }
        }
    }
}
```

Run it and inspect Console Output. The hostname should be the factory laptop/PC. If the job runs on another machine, check the label configuration. This confirms Jenkins can schedule a basic command on the factory agent.

5. Integrate it with WIC HSIT safely

Step 7 — Confirm target access on the factory machine

1. Identify the real interface: for example, debugger/JTAG/USB through CCS, Ethernet, serial, or a test instrument. Confirm the exact setup with the hardware team.
1. Install approved drivers and tools using the approved installation process and license arrangements.
1. Run the target discovery/connect procedure manually on the factory laptop first.
1. Verify the expected target identity/configuration so a test cannot accidentally operate the wrong board.
1. Run a harmless diagnostic, then one approved HSIT case with a defined expected result.

Step 8 — Decide how code and test inputs reach the factory

Option	When appropriate	Decision to confirm
Factory agent checks out from SVN	Only if the factory subnet and account are explicitly authorized for SVN access.	SVN routing, credentials, repository permissions and revision pinning.
Controller checks out and transfers a package	Useful when SVN should remain restricted to the development location.	Approved transfer path, package integrity, exact revision metadata and access control.
Controlled offline hand-off	If the networks cannot exchange data directly or policy requires separation.	Approved removable media/process, checksums, chain of custody and evidence transfer.

For the initial design, investigate controller checkout plus an approved package hand-off first. It makes the tested revision explicit and avoids assuming the factory agent can access SVN. This is a proposal, not a decision to bypass network policy.

Step 9 — Add a separate HSIT stage/job

1. Keep the existing working LDRA pipeline unchanged until the proof of concept passes.
1. Create a dedicated downstream job or stage constrained to `wic-hsit`.
1. Pass a known software revision/build identifier and the approved test configuration to the factory job.
1. Ensure only one job can operate the target at a time; keep one executor unless the hardware/test design explicitly supports concurrency.
1. Run the approved test script, capture raw output and machine-readable results where available, and archive the evidence.
1. Return a clear status to the controller: PASS only when all required checks for the executed test pass; otherwise FAIL, UNSTABLE, BLOCKED or NOT RUN according to the team's defined status rules.

Step 10 — Consolidated report

1. Project and software revision / package identifier.
1. Build and test configuration, tool versions, target identity and test procedure revision.
1. Static-analysis findings and disposition; unit-test results; SSIT results where applicable; HSIT test-case results.
1. Expected result versus actual result for each executed test, plus logs and any captured measurements.
1. Failures, unavailable target, skipped tests, environment issues and unresolved findings.
1. Report timestamp and links to archived evidence; reviewer/approval fields as required by the certification process.

6. Questions the manager may ask

“If we can ping the target, why do we need an agent?”

Ping checks limited network reachability. It does not provide a job scheduler, install the target tools, launch a test, collect evidence or compare actual results with expected results. An agent lets Jenkins execute a job on the factory PC that has local access to the hardware. Ping can still be one diagnostic if the target supports Ethernet/ICMP.

“Does the Jenkins Agent connect directly to the hardware?”

Not necessarily. The agent is software running on the factory PC. The PC connects to the target using the actual interface—such as CCS plus a debugger/JTAG connection, Ethernet, serial or approved test equipment. Jenkins talks to the agent; the local tools talk to the target.

“What if the two subnets are different?”

Different subnets can communicate through approved routing and firewall rules. If inbound connections from the controller are restricted, an inbound agent may initiate the connection from the factory side. The network team must approve the required route and protocol. An agent cannot bypass an isolated network.

“What if ping fails?”

Do not conclude immediately that Jenkins cannot work. ICMP may be blocked. Test the approved Jenkins URL/connection and the local target interface separately. If the target uses JTAG/USB rather than Ethernet, pinging the target may not apply.

“What if the agent is online but HSIT fails?”

Agent-online status proves only that Jenkins can communicate with the agent. Check the target discovery, driver/debugger logs, software load, test script, expected-result checks and test environment. Keep connection failures distinct from functional test failures.

“Can all testing be performed from our location?”

Static analysis and target-independent tests can usually run in the development environment. Hardware-dependent tests need the target or an equivalent approved environment. Running them remotely is feasible if a factory-side agent has access to the target and a permitted connection to Jenkins.

“Can we say the code has no violations?”

Only within the scope of the checks that were actually executed and passed. Static-analysis findings, unit-test results, integration tests and HSIT evidence are different categories. A report must list what ran, what passed, what failed, what was skipped and any unresolved findings. Certification wording must follow the approved plan and review process.

“Why use a laptop rather than moving the target to us?”

The laptop can stay beside the physical target and provide local access to its debugger and test interfaces. It avoids repeatedly moving the hardware. The trade-off is that the laptop becomes test infrastructure and needs controlled access, stable power/network, maintenance, compatible tool versions and a reliable startup/reconnection procedure.

7. Risks, checks and proposed milestones

Milestone	Evidence required before proceeding
1. Network feasibility	Approved Jenkins URL reachable from factory machine; network team confirms allowed agent connection method and rules.
2. Agent proof of concept	Node stays online; a label-restricted job prints the factory machine hostname and exits successfully.
3. Tool readiness	Approved CCS/debugger/driver/tool versions installed and verified; licenses and account permissions confirmed.
4. Target readiness	Correct target identified and connected; harmless diagnostic succeeds.
5. Single HSIT case	One approved test runs; expected result is checked; raw logs and metadata are retained.
6. Pipeline integration	Known revision and test inputs reach the agent; status and evidence return to the controller.
7. Reliability and evidence	Restart/reconnect, target-unavailable behavior, timeout, failure notification and report completeness are tested.
8. Certification review	Test scope, traceability, evidence format and sign-off match the approved project/certification process.

Security and reliability safeguards

1. Use an approved dedicated machine and least-privilege account; limit which jobs can run on the agent.
1. Protect the agent secret and credentials. Do not store secrets in source control or report files.
1. Do not open broad firewall access; request only the approved required route/protocol.
1. Do not allow multiple jobs to control the same physical target concurrently.
1. Make target unavailability, timeout, missing log and unexecuted tests visible in the report.
1. Keep test results tied to the exact revision and test configuration.
1. Do not deploy code to operational/production hardware or run destructive tests without explicit authorization and an approved procedure.

Proposed action list for tomorrow

1. Present the factory-side inbound agent concept and the three-layer distinction: Jenkins connectivity, local target connectivity, actual test verification.
1. Ask the manager to nominate the factory laptop/PC and hardware/test contact.
1. Ask the network/IT team to confirm permitted controller URL, agent connection method, routing/firewall rules and security requirements.
1. Confirm whether target access is via CCS/debugger/JTAG, Ethernet or another interface.
1. Agree on SVN/package transfer method and a known test revision.
1. Request permission to perform a non-destructive proof of concept before changing the existing pipeline.

8. One-minute summary and references

One-minute summary to memorize

“My proposal is to keep our current Jenkins Controller and LDRA workflow at our location, and use a dedicated Windows laptop/PC at the factory as a Jenkins inbound Agent. Jenkins will schedule the HSIT job on that labelled agent. The factory agent will use the locally installed approved debugger/test tools to operate the WIC target, check actual outputs against expected results, and return logs and test evidence to Jenkins. Ping is useful only as a limited network diagnostic: it does not prove the Jenkins connection, target-interface access or functional test success. The next steps are to confirm the network rules and target interface, bring the agent online, run a harmless job, verify target access, execute one approved HSIT case, and only then integrate it with the existing pipeline.”

Official Jenkins documentation

1. Managing Nodes — Jenkins documentation
Explains nodes/agents, inbound agent setup and Windows startup approaches.

2. Using Jenkins Agents
Explains controller/agent roles, labels, executors and job placement.

3. Pipeline Syntax
Explains the `agent { label ... }` directive used to run a stage on a specific node.

4. Using a Jenkinsfile
Explains pipeline structure and pipeline-as-code.

Assumptions and limitations

This document is a proposed architecture and proof-of-concept plan, not an approved BEL network design or certification procedure. The actual Jenkins version, agent launch options, Java compatibility, target interface, CCS/debugger configuration, test scripts, licenses, security controls and required evidence must be confirmed locally. Use the agent command generated by your own Jenkins node page rather than copying a placeholder command from this document.

Prepared: 12 October 2026